Prompt: This is the test.cpp file generated by chatgpt for the game based solely on the code files and readme. However, we needed to follow the test convention as outlined by µnit — C Unit Testing Framework
First, can you explain what a test framework is? Is it just a set of conventions/procedures to use to define test cases?
Second, how and why is the chatgpt generated test.cpp file different/incompatible with the muunit C testing framework? What does the file contain, and what is it even doing?
Third, I imported a .yml file into the repo for the purposes of demonstrating our implementation of build systems. However, it now causes an error to appear in the github repo webpage (not catastrophic, I can still run and play the game when I fetch it onto my machine). Specifically, the error is shown in the attached image. Attached also is that .yml file. Why?

Yes. I can already see one likely reason for the GitHub failure from the YAML file alone.

1. What is a testing framework?

Your intuition is basically correct.

A testing framework is:

- A set of conventions for organizing tests.
- A library that provides assertion functions.
- A test runner that executes tests and reports failures.
- Sometimes utilities for setup/teardown and grouping tests.

Without a framework, you might write:

C++

int main() {

if (add(2,3) != 5) {

std::cerr << "Test failed\n";

return 1;

}

}

A framework gives you standard tools instead:

C++

MunitResult test_add(const MunitParameter params[], void* data) {

munit_assert_int(add(2,3), ==, 5);

return MUNIT_OK;

}

and then the framework:

- discovers tests,
- runs them,
- counts passes/failures,
- prints standardized reports.

Think of it as the difference between:

"I wrote my own loop to check results"

and

"I plugged my tests into an existing testing infrastructure."

For your coursework, they specifically requested **µnit**, so they probably wanted students to demonstrate using an existing testing framework rather than inventing a homemade one.

---

2. What is ChatGPT's test.cpp actually doing?

The file is essentially a **custom test runner**. It is **not** a µnit test suite. [onedrive.live.com]

The structure is:

A) Custom assertion function

C++

void check(bool condition, const char* message) {

if (!condition)

throw std::runtime_error(message);

}

This is acting like a very primitive version of:

C++

munit_assert(...)

in µnit. [onedrive.live.com]

B) Fake keyboard input / output capture

The class:

C++

struct IOCapture

redirects:

C++

std::cin

std::cout

into string streams. [onedrive.live.com]

This allows a test to pretend a user typed:

Plain Text

3

1 1

2 1

without requiring somebody to sit at the keyboard.

For example:

C++

IOCapture io("10\n");

check(readPlayerCount() == 10);

feeds "10" into readPlayerCount(). [onedrive.live.com]

That's actually a clever and legitimate testing technique.

C) A collection of test functions

Examples:

C++

void test_board_full()

C++

void test_winning_lines()

C++

void test_complete_program()

Each function performs checks and throws an exception if something fails. [onedrive.live.com]

D) A handmade test runner

At the bottom:

C++

const Test tests[] = { ... };

stores all tests.

Then:

C++

for (const Test& test : tests)

runs them one by one. [onedrive.live.com]

Success:

C++

PASS: board full

Failure:

C++

FAIL: board full

So why is it incompatible with µnit?

Because µnit expects a completely different structure.

µnit tests look roughly like:

C

```
static MunitResult test_board_full(
const MunitParameter params[],
void* fixture)
{
munit_assert(...);
return MUNIT_OK;
}
```

and are registered like:

C

```
static MunitTest tests[] = {
{
"/board_full",
test_board_full,
...
},
{ NULL, NULL, NULL, NULL, MUNIT_TEST_OPTION_NONE, NULL }
};
```

and executed by:

C

```
return munit_suite_main(...);
```

The ChatGPT file instead:

- uses exceptions,
- uses a custom check(),
- implements its own runner,
- knows nothing about Munit types,
- never includes munit.h.

Therefore, although it performs testing, it is **not using the framework required by the assignment**. [onedrive.live.com]

---

3. Is the test file completely wrong?

Not necessarily.

The ideas are mostly reasonable:

✅ Test board initialization

✅ Test winner detection

✅ Test draw detection

✅ Test invalid input

✅ Test full game execution

✅ Simulate user input

Those are all things I would expect in a decent test suite.

The problem is not the tests themselves.

The problem is that they're implemented as a homemade framework instead of µnit.

A good strategy would be:

1. Keep many of these test ideas.
2. Rewrite them into µnit test functions.
3. Replace

C++

check(...)

with

C++

munit_assert(...)

4. Register them in the µnit test table.

---

4. Why is GitHub Actions failing?

I already see a likely culprit:

YAML

uses: actions/checkout@v7

[onedrive.live.com]

As far as GitHub Actions releases go, the commonly used versions are:

YAML

actions/checkout@v4

not v7.

If GitHub cannot find that action version, the workflow immediately fails before compiling anything.

Other possible issue

Your workflow compiles:

YAML

tests/Tic_Tac_Two_tests_for_main.cpp

[onedrive.live.com]

If that file does not exist exactly at:

Plain Text

tests/

Tic_Tac_Two_tests_for_main.cpp

GitHub's runner will fail with something like:

Plain Text

fatal error:

tests/Tic_Tac_Two_tests_for_main.cpp:

No such file or directory

Other possible issue

The test file references many symbols such as:

C++

readPlayerCount()

initBoard()

printBoard()

parseBoard()

gameLoop()

[onedrive.live.com]

but your compile command is:

YAML

g++ ... tests/Tic_Tac_Two_tests_for_main.cpp -o game_tests

[onedrive.live.com]

Notice what's missing:

Plain Text

main.cpp

or

Plain Text

game.cpp

or whatever actually defines those functions.

If the test file doesn't #include the implementation, you'll get linker errors such as:

Plain Text

undefined reference to readPlayerCount

undefined reference to gameLoop

What I'd do next

Click **Details** on the failed GitHub Action and send me the full build log (or a screenshot of the red error section).

The build log will tell us exactly which of these is happening:

1. Invalid GitHub Action version (checkout@v7)
2. Wrong file path
3. Missing source files during compilation
4. Missing µnit library/header
5. Some other compiler error

My guess from the YAML alone is that the first thing to check is:

YAML

uses: actions/checkout@v4

instead of

YAML

uses: actions/checkout@v7

but the build log will let us identify the exact failure in one step.

Prompt: This is the output of the error on github.

Ah, this screenshot is actually much more informative than the first one.

The failure is **not coming from your test.cpp file at all**.

The key lines are:

Plain Text

Run ./configure

./configure: No such file or directory

Error: Process completed with exit code 127.

What GitHub is trying to do

Notice the sequence:

Plain Text

✓ actions/checkout@v4

✗ configure

○ make

○ make check

○ make distcheck

GitHub is attempting to run the classic GNU/Autotools workflow:

Shell

./configure

make

make check

make distcheck

This is the sort of workflow used by projects that contain files such as:

Plain Text

configure

configure.ac

Makefile.am

The first step:

Shell

./configure

is expected to generate a Makefile.

But GitHub reports:

Plain Text

./configure: No such file or directory

which means there is no executable file called configure in the repository root.

Why is GitHub doing this?

This is the interesting part.

Your workflow file that you showed me earlier was:

YAML

steps:

- uses: actions/checkout@v4

- name: Compile tests

run: g++ ...

That workflow would **not** run:

Shell

./configure

at all.

Yet the screenshot clearly shows:

Plain Text

configure

make

make check

make distcheck

which means GitHub is running a **different workflow**.

Most likely one of the following:

Possibility 1

There is another workflow file already in:

Plain Text

.github/workflows/

For example:

Plain Text

ci.yml

build.yml

autotools.yml

and that workflow is being triggered instead.

Possibility 2

One of your teammates added a workflow that assumes an Autotools project.

Something like:

YAML

- name: configure

run: ./configure

- name: make

run: make

- name: make check

run: make check

If your project does not use Autotools, this will immediately fail.

What should a simple coursework workflow look like?

For your tic-tac-toe project, if you're just compiling one source file, it would probably look more like:

YAML

name: Build

on:

push:

pull_request:

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Compile

run: g++ -std=c++17 -Wall -Wextra main.cpp -o game

- name: Run

run: ./game << EOF

3

EOF

or similar.

No:

Shell

./configure

make

required.

How to verify immediately

Look in your repository under:

Plain Text

.github/

└── workflows/

and tell me **every file that exists there**.

For example:

Plain Text

.github/workflows/ci.yml

.github/workflows/build.yml

.github/workflows/test.yml

The workflow shown in the screenshot is almost certainly contained in one of those files.

One more observation

Notice that GitHub successfully executed:

Plain Text

actions/checkout@v4

This means my earlier suspicion about checkout@v7 was wrong.

The workflow that actually ran is already using:

YAML

actions/checkout@v4

which reinforces the conclusion that the workflow being executed is **not the same YAML file** you showed me earlier. The screenshot and the YAML contents do not match each other.

My current best guess

You now have **at least two workflow files** in .github/workflows/:

1. Your new ci.yml
2. An older workflow that runs:

Shell

./configure

make

make check

make distcheck

The older workflow is failing because your repository does not contain a configure script.

If you show me the contents of .github/workflows/ (or upload the files), I can tell you exactly which workflow GitHub is executing and which one should be removed or modified.

Prompt: Our repo actually has a .github/workflows/ with c-cpp.yml. In the main repo (not inside the .github/workflows folder) we also have ci.yml.
To obtain this build thing, on github, I hit new workflow and selected C/C++ with Make, which states "Build and test a C/C++ project using Make." It created a .yml file itself.

That explains everything.

What happened is that GitHub generated a workflow template for a **Make/Autotools project**, but your tic-tac-toe game is just a C++ source file (or a few source files) and does **not** use Make or Autotools.

The generated workflow is essentially assuming your repository looks like:

Plain Text

configure

Makefile

src/

tests/

and can be built via:

Shell

./configure

make

make check

But your repository looks more like:

Plain Text

main.cpp

README.md

.github/workflows/c-cpp.yml

(or similar)

so GitHub runs:

Shell

./configure

and immediately fails because no such file exists.

Why c-cpp.yml is causing the failure

The important detail is this:

GitHub Actions only executes workflow files inside .github/workflows/.

Your file:

Plain Text

.github/workflows/c-cpp.yml

is being executed.

Your file:

Plain Text

ci.yml

in the repository root is just an ordinary file. GitHub Actions completely ignores it.

So the failing workflow is almost certainly this auto-generated c-cpp.yml.

Why GitHub generated it

When you selected:

"C/C++ with Make"

GitHub assumed:

"This project uses a Makefile."

That's a pretty common build system in software engineering.

The generated template therefore includes steps such as:

YAML

- run: ./configure

- run: make

- run: make check

which are the standard commands for a Make/Autotools project.

For coursework purposes, that's actually useful because it demonstrates CI/CD concepts, but only if the repository is structured accordingly.

What should you do?

Option 1: Remove the workflow entirely

If your coursework only requires demonstrating that you know what a workflow is, you could simply delete:

Plain Text

.github/workflows/c-cpp.yml

and the red Xs disappear.

Option 2: Replace it with a simple compile workflow

Suppose your whole game is in:

Plain Text

main.cpp

Then:

YAML

name: C++ Build

on:

push:

pull_request:

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Compile

run: g++ -std=c++17 -Wall -Wextra main.cpp -o game

This would actually compile your game whenever somebody pushes code.

Option 3: Build and run tests

If you later rewrite your tests using μnit, you could do:

YAML

- name: Compile tests

run: g++ test.cpp main.cpp -o tests

- name: Run tests

run: ./tests

and GitHub would automatically execute the tests on every push.

This is actually the sort of thing lecturers usually want when they talk about:

- build systems
- unit testing
- continuous integration

Why the screenshot shows "configure", "make", and "make check"

Those names correspond to the workflow steps in c-cpp.yml.

GitHub isn't magically deciding to do those things.

It is literally reading commands from that YAML file and executing them:

YAML

run: ./configure

run: make

run: make check

The first command fails:

Plain Text

./configure: No such file or directory

so the remaining steps are skipped.

One subtle point

You mentioned that you imported ci.yml to demonstrate build systems.

At the moment, that file is not demonstrating anything to GitHub because it's in the repository root.

For GitHub Actions to execute it, it must be located in:

Plain Text

.github/workflows/ci.yml

Only files inside that folder are treated as workflows.

So right now:

Plain Text

.github/workflows/c-cpp.yml

✅ Executed

Plain Text

ci.yml

❌ Ignored

If you paste the contents of c-cpp.yml, I can tell you exactly which lines should be removed and how to convert it into a minimal workflow appropriate for your coursework project.

Prompt: This is the c-cpp.yml file inside the workflows folder:
name: C/C++ CI
on: push: branches: ["main"] pull_request: branches: ["main"]
jobs: build:
runs-on: ubuntu-latest
steps: - uses: actions/checkout@v4 - name: configure run: ./configure - name: make run: make - name: make check run: make check - name: make distcheck run: make distcheck

This is the ci.yml file outside that folder:
name: C++ tests
on: [push, pull_request]
permissions: contents: read
jobs: test: runs-on: ubuntu-latest steps: - name: Check out repository uses: actions/checkout@v7 - name: Compile tests run: g++ -std=c++17 -Wall -Wextra -pedantic -I. tests/Tic_Tac_Two_tests_for_main.cpp -o game_tests - name: Run tests
run: ./game_tests
Note that our repo now contains the files as shown in the attached screenshot, where test_last.cpp is an updated version of a now deleted test.cpp; it is the same code except written to follow the munit testing conventions.

Now we can see exactly what's happening.

There are actually **two separate issues**.

Issue 1: The red X on GitHub right now

The red X is caused by:

YAML

.github/workflows/c-cpp.yml

not by your tests.

The workflow says:

YAML

- name: configure

run: ./configure

- name: make

run: make

but your repository does not contain:

Plain Text

configure

Makefile

so GitHub executes

Shell

./configure

and gets

Plain Text

No such file or directory

which is precisely the error shown in the screenshot.

In other words:

GitHub is failing before it even reaches any game code, test code, or μnit code.

The simplest fix would be either:

Remove the workflow

Delete:

Plain Text

.github/workflows/c-cpp.yml

if you only wanted to show that you explored Actions.

or

Replace the workflow

Change it to compile your actual game files instead of trying to run Autotools commands.

Issue 2: Your ci.yml is currently ignored

GitHub Actions only executes workflow files located in:

Plain Text

.github/workflows/

Currently you have:

Plain Text

ci.yml

in the repository root.

GitHub treats that as an ordinary file.

It never runs.

So:

Plain Text

.github/workflows/c-cpp.yml

✅ executed

Plain Text

ci.yml

❌ ignored

There is also a problem with your ci.yml

Even if GitHub were executing it, I can already see a few errors.

Problem A

YAML

uses: actions/checkout@v7

There is no checkout v7.

Use:

YAML

uses: actions/checkout@v4

instead.

Problem B

The compile command is

YAML

g++ -std=c++17 -Wall -Wextra -pedantic -I. \

tests/Tic_Tac_Two_tests_for_main.cpp \

-o game_tests

But your repository screenshot shows:

Plain Text

test_last.cpp

and not

Plain Text

tests/Tic_Tac_Two_tests_for_main.cpp

So this would fail with something like:

Plain Text

No such file or directory

Problem C

I see:

Plain Text

game.cpp

game.h

main.cpp

test_last.cpp

which suggests your tests probably need to be linked against the game code.

Something more like:

YAML

run: g++ -std=c++17 \

test_last.cpp \

game.cpp \

-o game_tests

would be needed.

Exactly what is required depends on how your μnit version is written.

Is test_last.cpp now closer to what the course wants?

Yes.

Based on your description:

"updated version of test.cpp that follows the munit testing conventions"

that is much more likely what the coursework expects.

Your original test.cpp was essentially:

Plain Text

Homemade assertions

+

Homemade test runner

+

Exception handling

The tests themselves were reasonable, but they weren't using the required framework.

If test_last.cpp now contains things like:

C++

#include "munit.h"

and

C++

MunitResult test_something(...)

and

C++

static MunitTest tests[]

then it is now acting as a proper μnit test suite rather than a custom framework.

What I would do if this were my coursework

I would delete the autogenerated Make workflow entirely and replace it with something like:

YAML

name: C++ Tests

on:

push:

pull_request:

jobs:

build:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v4

- name: Compile

run: g++ -std=c++17 -Wall -Wextra -pedantic test_last.cpp game.cpp -o game_tests

- name: Run tests

run: ./game_tests

That would actually demonstrate:

- GitHub Actions
- CI
- automatic builds
- automatic test execution

which is usually exactly what software engineering instructors want to see.

One thing I'd still like to inspect is the contents of test_last.cpp. If you paste that file, I can tell you whether it is genuinely using μnit correctly and what compile command the workflow should use.